单片机考试题型：

诚信应考，闭卷考试，满分 100 分，共四大题。

一、单项选择题（共 15 小题，每小题 2 分，共 30 分）

二、判断题（共 15 小题，每小题 1 分，共 15 分）

三、简答题（共 5 小题，共 25 分）

四、程序分析与编程题（共 2 小题，共 30 分）

单片机-第 1 章练习题

主要考查的知识点：嵌入式系统的组成及应用。

一、单选题

1、以下哪种设备不属于嵌入式系统？（ B ）

A、智能手机　　B、通用 PC 机　　C、工业机器人　　D、心电监护仪

解析：通用 PC 机不属于嵌入式系统定义范畴。

2、嵌入式系统的行业普遍定义中，不包含以下哪个要点？（ B ）

A、 以应用为中心　　B、 软、硬件不可剪裁

C、 以计算机技术为基础　　D、 适合严格要求的专用计算机系统

解析：嵌入式系统软、硬件可剪裁。

3、智能手环中，负责处理数据和控制各模块的是（ C ）

A、蓝牙模块　　B、加速度传感器　　C、微控制器　　D、OLED 显示屏

解析：微控制器负责处理数据和控制各模块。

4、嵌入式系统硬件中，MCU 不包含以下哪个部分？（ B ）

A、 运算器　　B、 键盘　　C、 控制器　　D、 存储器

解析：键盘属于外围电路，不属于 MCU。

5、四旋翼无人机中，微控制器的主要任务不包括（ B ）

A、 导航　　B、 充电　　C、 通信　　D、 控制

解析：四旋翼无人机中微控制器不负责充电。

二、简答题

1、简述嵌入式系统的组成部分及其各自的作用。

参考答案：参考答案：嵌入式系统主要由硬件和软件两部分组成。硬件部分主要包括最小系统和外围电路两部分，最小系统由嵌入式处理器、电源电路、时钟电路、复位电路和调试电路等组成；外围电路包括用于人机交互的显示电路和键盘电路、用于数据采集的采集电路和用于信息传输的通信电路等，由具体的工程应用决定。软件部分由驱动程序、操作系统(可选) 和应用程序三部分组成。驱动程序提供硬件和软件之间的接口函数；操作系统负责进程调度和资源管理；应用程序实现具体的业务逻辑，由开发人员根据具体的项目需要进行编写。

2、分析新应用中 “MCU + RF + Sense + Algorithm” 各部分的含义及在智能手环中的应用体现。

参考答案：“MCU”是微控制器，在智能手环中负责整体控制和数据处理；“RF” 是射频模块，智能手环中的蓝牙模块属于此类，用于无线通信，实现与手机等设备同步数据；“Sense”是传感部分，如智能手环中的心率传感器、加速度传感器，用于采集运动、心率等数据；“Algorithm”是算法，智能手环利用算法对采集的数据进行分析，如分析运动数据判断运动状态、分析睡眠数据评估睡眠质量等。

单片机--第 2 章练习题

主要考查的知识点：STM32 最小系统的组成部分；通过查表可以知道具体芯片型号代表的含义。

一、单选题

1、STM32 是（ B ）位的单片机。

A、16　　B、32　　C、64　　D、128

2、STM32F1 系列采用的是什么内核？（ B ）

A、Cortex-M0　　B、Cortex-M3　　C、 Cortex-M4　　D、Cortex-M7

3、STM32F103ZET6 芯片中，“Z”代表什么含义？（ A ）

A、 芯片有 144 管脚　　B、512K 字节的 Flash

C、采用 LQFP 封装　　D、工作温度范围为- 40–+85℃

4、**STM32F103** 系列微控制器中，复位引脚是（ A ）。

A、 NRST　　B、 BOOT0　　C、 BOOT1　　D、 SWD

5、**STM32** 最小系统电路不包括以下哪个部分？（ B ）

A、电源电路　　B、显示电路　　C、时钟电路　　D、复位电路

二、判断题

1、ARM 公司既设计芯片内核，也生产芯片。（ x ）

解析：ARM 公司设计芯片内核，但不生产芯片，而是出售芯片技术授权

2、**STM32F1** 系列微控制器的所有引脚功能都是固定的，不能更改。（ x ）

解析：STM32 的部分引脚功能可复用，不是固定不变的

3、在 **STM32** 的启动模式中，主闪存存储器被选为启动区域时是正常启动模式。（ √ ）

4、计算机系统中，**CPU** 的运行速度是最快的，其次是存储器，外设运行速度最慢。（ √ ）

5、**STM32** 的每个 **GPIO** 引脚具备多种功能，这些功能可以同时使用。（ × ）

三、简答题

1、简述 STM32 芯片命名规范中 STM32F103VET6 各部分的含义。

参考答案：“STM32” 表示 ARM Cortex - M3 内核的 32 位微控制器；“F”代表芯片系列；“103”是增强型系列；“V”表示芯片有 100 管脚；“E”表示 512K 字节的 Flash；“T”表示采用 LQFP 封装；“6”表示工作温度范围为 -40~+85℃。

2、请列举 STM32 最小系统电路的组成部分，并简述每个部分的作用。

参考答案：STM32 最小系统电路由电源电路、时钟电路、复位电路、调试电路和主控芯片组成。电源电路为整个电路系统提供所需的供电电压；时钟电路为 MCU 提供时钟信号，确定各外设的工作速度；复位电路为 MCU 提供统一的初始状态；调试电路为 MCU 提供程序下载和调试的接口；主控芯片是核心，负责控制和处理数据。

单片机--第 3 章练习题

主要考查的知识点：STM32CubeMX 软件新建工程的基本步骤、程序下载以及调试过程中常遇到的问题及解决方法。

一、单选题

1、STM32CubeMX 是基于什么环境运行的？（ B ）

A、C++ 环境　　B、Java 环境　　C、 Python 环境　　D、C#环境

解析：STM32CubeMX 软件是基于 Java 环境运行的，需要先安装 JRE（Java Run Time Environment）。

2、在使用 STM32CubeMX 生成 MDK-Keil5 工程时，若要配置高速外部时钟 HSE，应选择（ C ）。

A、不设置　　B、内部时钟

C、Crystal/Ceramic Resonator　　D、随便选一个

解析：配置 RCC 时，选择高速外部时钟 HSE 为“Crystal/Ceramic Resonator”。

3、以下哪个不是嵌入式开发的集成开发环境？（ C ）

A、Keil5 MDK　　B、STM32CubeMX

C、Visual Basic　　D、STM32CubeIDE

解析：针对 ARM 内核的主流集成开发环境主要有 Keil5 MDK、STM32CubeMX 和 STM32CubeIDE，Visual Basic 不是用于 STM32 开发的集成开发环境。

4、在 Keil5 软件安装时，安装路径最好（ B ）。

A、包含中文　　B、不包含中文　　C、无所谓　　D、放在系统盘根目录

5、STM32CubeMX 生成的工程中，用于处理中断事件的文件是（ C ）。

A、main.c　　B、gpio.c

C、stm32f1xx_it.c　　D、stm32f1xx_hal_msp.c

解析：stm32f1xx_it.c 是中断处理文件，包含各种中断服务函数，用于处理微控制器接收到的中断事件。

二、简答题

1、请简述使用 STM32CubeMX 和 Keil5 开发 STM32 项目的完整流程。

参考答案：

（1）打开 STM32CubeMX 软件新建 Project 工程，选择 MCU（如 STM32F103VET6）。

（2)配置 RCC(选择高速外部时钟 HSE 为“Crystal/Ceramic Resonator”)、调试接口模块（如 Serial Wire）、GPIO（设置相应引脚为输出模式等）、时钟树（设置 HCLK 频率等）。

（3）设置工程相关参数（工程名称、存放位置），在 Code Generator 页面设置生成代码特性，单击“GENERATE CODE”生成工程代码。

（4）使用 Keil5 打开生成的工程，编写用户程序，然后进行编译链接，生成.hex 文件。

（5）连接开发板，在“Debug”选项卡中选择“ST-Link Debugger”，单击 Settings 按钮，勾选中“Reset and Run”，擦除的 FLASH 选择“Erase Sectors”，最后进行程序下载和调试。

2、在使用 Keil5 进行程序下载和调试时，如果出现下载失败的情况，请分析可能的原因并给出解决方法（列举 3 种即可）。

参考答案：

（1）工程代码问题：工程代码中可能存在语法错误或与硬件不匹配的配置。重新编译工程，查看编译输出窗口的错误提示，修改代码中的语法错误；检查在 STM32CubeMX 中生成的工程代码配置是否与实际硬件一致，如引脚配置、时钟配置等。

（2）驱动问题：可能 ST-Link 驱动未正确安装。解决方法是重新安装 ST-Link 驱动，确保安装过程正确，安装完成后在设备管理器中查看驱动是否正常识别。

（3）连接问题：开发板与电脑连接不稳定或连接线路损坏。检查开发板与电脑的连接线路，确保连接牢固，尝试更换 USB 线；查看开发板电源是否正常供电，确保开发板处于正常工作状态。

（4）Keil5 设置问题：在 Keil5 中“Debug”选项卡的设置可能有误。重新检查“Debug”选项卡中是否选择了正确的调试工具（如 ST-Link Debugger），Settings 中的“Reset and Run”是否勾选，擦除的 FLASH 选择是否正确（如选择“Erase Sectors”）。

3、在使用 STM32CubeMX 软件生成的 Keil5 工程中，程序编译成功但下载到目标板后无预期现象（如 LED 未点亮、按键按了没反应、串口无输出等），请分析可能的原因并提供解决办法（列举 3 种即可）。

参考答案：（1）硬件问题：目标板供电异常（电压不足或不稳）、引脚连接错误、SWD/JTAG 调试接口连接异常导致程序未正确下载或运行。解决方法：检查电源电压（通常为 3.3V），确认目标板供电正常。核对 CubeMX 中的引脚配置与硬件连接是否一致，重新插拔连接线。在 Keil5 中使用调试器单步运行程序，观察是否执行到预期代码位置。

（2）CubeMX 配置错误：外设引脚模式配置错误（如 GPIO 未设置为输出）、时钟源配置与硬件晶振不匹配、芯片型号选择错误。解决方法：在 CubeMX 中重新检查引脚模式、时钟树配置（确保与硬件晶振频率一致），并确认芯片型号正确。重新生成代码，替换工程中的原有文件。

（3）用户程序逻辑错误：外设初始化失败、中断未正确配置、主程序逻辑错误（如死循环或条件判断错误）。解决方法：在 main()函数或关键代码段添加断点，通过单步调试定位问题。检查外设初始化函数（如 HAL_GPIO_Init()）的返回值，确认初始化成功。若使用中断，确保中断优先级和使能设置正确。

STM32 微控制器练习题

主要考查的知识点：哈佛结构和冯诺依曼结构的去呗、STM32 的系统结构、STM32 的存储结构和时钟系统，本小节主要以了解为主，内容考的不多，考试中主要出现在选择题和判断题中。

一、单选题

1、以下哪个是低速外设总线？（ **B** ）

A、AHB　　B、APB1　　C、APB2　　D、System

2、**STM32** 系统架构中，专门用来取指的总线是（ **B** ）。

A、 ICode 总线　　B、DCode 总线　　C、系统总线　　D、DMA 总线

解析：DCode 总线是用来取数的，变量存放在内部 SRAM 中，通过 DCode 总线访问，所以选 B。

3、**STM32** 的片上外设挂载在（ **B** ）。

A、AHB 总线　　B、APB1 和 APB2 总线

C、FSMC 总线　　D、DMA 总线

解析：从 AHB 总线延伸出来的 APB1 和 APB2 总线上挂载着 STM32 的各种特色外设，如 GPIO、串口、I2C、SPI 等，所以选 B。

4、**STM32** 总的地址空间大小为（ **C** ）。

A、1GB　　B、2GB　　C、4GB　　D、8GB

5、**STM32** 的时钟系统中，高速内部时钟 **HSI** 的频率为（ **B** ）。

A、4MHz　　B、8MHz　　C、16MHz　　D、32MHz

解析：高速内部时钟 HSI 由内部 RC 振荡器产生，频率为 8MHz

二、判断题

1、**STM32** 的存储结构中，**4GB** 地址空间被平均分成 **4** 个块，每块 **1GB**。（ x ）

解析：4GB 地址空间被平均分成 8 个块，每块 512MB。

2、**STM32** 的高速外部时钟 **HSE** 必须采用 **8MHz** 的晶振。（ x ）

解析：HSE 晶振频率可取 4 - 16MHz，一般采用 8MHz，但不是必须采用 8MHz。

3、**STM32** 的锁相环 **PLL** 只能用于倍频 **HSI**。（ x ）

解析：PLL 可用于倍频 HSI 或 HSE，保证外部输入时钟信号与内部振荡信号同步。

4、**STM32** 的 **MCO** 引脚输出的时钟频率是固定的。（ x ）

解析：MCO 的时钟来源可以是 PLLCLK/2、HSI、HSE、SYSCLK，所以其输出时钟频率不固定。

5、对 STM32 外设的操作和对存储器的操作不一样。（ x ）

解析：STM32 的外设可以通过指针来实现访问和操作，每个外设寄存器都占用特定的存储器地址，对外设的操作和对存储器的操作是一样的。

6、STM32 的内部闪存存储器（FLASH）用于存储程序变量。（ x ）

解析：内部闪存存储器（FLASH）用于存储编写好的程序，程序变量存储在内部 SRAM 中。

7、STM32 系统架构中，系统总线主要用于传输数据。（ x ）

解析：系统总线主要用于访问外设的寄存器，DMA 总线主要用于传输数据。

8、Cortex-M3 是 32 位内核，存储采用哈佛结构。(√)

9、NRST 复位引脚，高电平有效。(×)

单片机--第 4 章 GPIO 练习题

主要考查的知识点：GPIO 的 8 种工作模式及特点、GPIO 的常用 HAL 库接口函数有哪些？对应实现什么功能。

一、单选题

1、以下哪种不是 GPIO 的工作模式？（ A ）

A、推挽输入模式　　B、浮空输入模式

C、开漏输出模式　　D、模拟输入模式

解析：GPIO 的工作模式有浮空输入、上拉输入、下拉输入、模拟输入、推挽输出、复用推挽输出、开漏输出、复用开漏输出，没有推挽输入模式，所以选 A。

2、当 GPIO 引脚配置为上拉输入模式时，无输入信号时引脚默认电平是（ A ）。

A、高电平　　B、低电平　　C、浮空　　D、不确定

解析：上拉输入模式下，引脚内部有上拉电阻连接到电源 VDD，无输入信号时默认输入高电平，所以选 A。

3、GPIO 的 HAL 库接口函数源码存放在哪个源文件中？（ B ）

A、stm32f1xx_hal_gpio.h　　B、stm32f1xx_hal_gpio.c

C、main.c　　D、gpio.c

解析：GPIO 的 HAL 库接口函数源码在源文件 stm32f1xx_hal_gpio.c 中，对应的头文件 stm32f1xx_hal_gpio.h 声明了这些函数。

4、HAL_GPIO_Init ( ) 函数的功能是（ B ）。

A、读取选定的端口引脚的电平状态　　B、 初始化指定引脚

C、设置选定的端口引脚输出高电平或低电平　D、翻转指定引脚的电平状态

5、若要读取 PA0 引脚的电平状态，应使用以下哪个函数？（ C ）

A、 HAL_GPIO_WritePin (GPIOA, GPIO_PIN_0, GPIO_PIN_SET)

B、 HAL_GPIO_TogglePin (GPIOA, GPIO_PIN_0)

C、 HAL_GPIO_ReadPin (GPIOA, GPIO_PIN_0)

D、 HAL_GPIO_Init (GPIOA, &gpio)

解析：HAL_GPIO_ReadPin ( )函数用于读取指定引脚的电平状态，所以读取 PA0 引脚电平应使用 HAL_GPIO_ReadPin (GPIOA, GPIO_PIN_0)，选 C。

6、在 HAL_GPIO_WritePin ( ) 函数中，若要使 PA12 引脚输出低电平，正确的调用方式是（ B ）。

A、HAL_GPIO_WritePin (GPIOA, GPIO_PIN_12, GPIO_PIN_SET)

B、HAL_GPIO_WritePin (GPIOA, GPIO_PIN_12, GPIO_PIN_RESET)

C、HAL_GPIO_WritePin (GPIOA, GPIO_PIN_12, 0)

D、HAL_GPIO_WritePin (GPIOA, GPIO_PIN_12, 1)

解析：HAL_GPIO_WritePin () 函数的第三个参数 GPIO_PIN_RESET 表示输出低电平，GPIO_PIN_SET 表示输出高电平，所以使 PA12 输出低电平应调用 HAL_GPIO_WritePin (GPIOA, GPIO_PIN_12, GPIO_PIN_RESET)，选 B。

7、下面关于 HAL_GPIO_TogglePin ( )函数的描述，正确的是（ C ）。

A、用于锁定引脚状态　　B、用于设置引脚为复用功能

C、用于翻转指定引脚的电平状态　　D、用于初始化引脚

8、STM32CubeMX 生成的工程中，用户应用程序代码应写在（ B ）。

A、任意位置

B、main.c 文件中的/* USER CODE BEGIN 3 /和/ USER CODE END 3 /之间

C、main 函数开头

D、gpio.c 文件中

9、在 STM32CubeMX 中配置时钟时，若采用 HSE 外部晶振 8MHz，要使系统时钟 SYSCLK 为 72MHz，PLL 的倍频系数应设置为（ B ）

A、 8　　B、 9　　C、 10　　D、 12

解析：采用 HSE 外部晶振 8MHz，通过 PLL 的 9 倍频可使系统时钟 SYSCLK 为 72MHz（8MHz×9 = 72MHz），所以选 B。

10、一个 STM32 的 GPIO 引脚作为复用 I/O 端口 AFIO 使用时，以下说法正确的是（ B ）。

A、可以同时使用多个复用功能　　B、一个时间段内只能使用一个复用功能

C、不能使用复用功能　　D、可随意切换复用功能

11、当 GPIO 引脚配置为模拟输入模式时，典型应用是（ C ）。

A、控制 LED 灯　　B、按键输入检测

C、A/D 模拟输入　　D、驱动电机

解析：模拟输入模式下，引脚信号连接到芯片内部的片上外设，典型应用是 A/D 模拟输入，对外部信号进行采集，所以选 C

12、在 HAL_GPIO_ReadPin ( ) 函数中，若读取到引脚为高电平，返回值是（ B ）。

A、GPIO_PIN_RESET　　B、GPIO_PIN_SET

C、0　　D、1

解析：HAL_GPIO_ReadPin () 函数返回值为 GPIO_PinState 枚举类型变量，GPIO_PIN_SET 表示读到高电平，GPIO_PIN_RESET 表示读到低电平，所以选 B。

13、在基于 HAL 库实现 LED 闪烁的实例中，HAL_Delay ( )函数的作用是（ A ）。

A、控制 LED 灯亮灭时间间隔　　B、初始化 LED 灯

C、配置 LED 灯引脚　　D、检测 LED 灯状态

解析：在 LED 闪烁实例中，HAL_Delay () 函数用于延时，控制 LED 灯亮灭的时间间隔，所以选 A。

14、GIPIO 挂载在 APB2 总线上，其对应的时钟频率为（ D ）

A、9MHZ　　B、36MHZ　　C、56MHZ　　D、72MHZ

二、判断题

1、**GPIO 只能作为通用输入/输出引脚使用，不能作为片上外设的 I/O 引脚。**（ x ）解析：GPIO 引脚除了作为通用输入/输出引脚使用外，还可以作为片上外设（如串口、ADC、IIC 等）的 I/O 引脚，所以该说法错误。

2、**STM32 微控制器中，每组 GPIO 端口都有 16 个引脚。**（ x ）

解析：STM32 微控制器每组 GPIO 端口最多有 16 个引脚，但不是所有组都一定是 16 个引脚，具体数量因芯片型号而异，所以该说法错误。

3、**GPIO 的输入模式包括上拉输入、下拉输入、浮空输入和模拟输出。**（ x ）

解析：GPIO 的输入模式包括上拉输入、下拉输入、浮空输入和模拟输入，不包括模拟输出，模拟输出不属于输入模式，所以该说法错误。

4、**HAL_GPIO_Init () 函数由用户调用，用于初始化指定引脚。**（ x ）

解析：HAL_GPIO_Init () 函数一般由 CubeMX 软件自动生成，不是由用户直接调用，所以该说法错误。

5、**复用推挽输出模式和推挽输出模式的区别在于输出高低电平的来源不同。**（ √ ）解析：复用推挽输出模式与推挽输出模式类似，区别在于输出高低电平的来源，复用推挽输出利用片上外设模块的复用功能输出，而推挽输出由 CPU 直接写输出数据寄存器决定，所以该说法正确。

6、**当 GPIO 引脚配置为开漏输出模式时，无需外接上拉电阻就能输出高电平。**（ x ）解析：开漏输出模式下，P - MOS 管一直处于截止状态，只有 N - MOS 管工作，只能输出低电平，需要外接上拉电阻，才能实现输出高电平，所以该说法错误。

三、简答题

1、试简述 STM32 的 GPIO 外设的 4 种输入模式和 4 种输出模式。

答：输入模式：上拉输入和下拉输入模式，浮空输入模式，模拟输入模式，输出模式：推挽输出模式，开漏输出模式，复用推挽输出，复用开漏输出

单片机--第 5 章 EXTI 练习题

主要考查的知识点：什么是中断？抢占优先级和响应优先级的区别、EXTI 外部中断的触发方式、EXTI 的常用 HAL 库接口函数有哪些？对应实现什么功能。

一、单选题

1、STM32 的所有端口都有外部中断能力，当使用外部中断线时，相应的引脚必须配置成（ B ）。

A、输出模式　　B、输入模式　　C、模拟输入模式　　D、复用输出模式

2、**EXTI 可以配置的触发方式不包括以下哪种？(D)**

A、 上升沿触发　　B、 下降沿触发

C、 双边沿触发　　D、 高电平触发

3、EXTI 的中断服务函数通常存放在哪个文件中？(B)

A、 main.c　B、 stm32f1xx_it.c　C、 hal_gpio.c　D、 nvic.c

解析：CubeMX 生成的中断服务函数默认在 *stm32f1xx_it.c* 中。

4、配置 PA0 引脚为 EXTI 中断线，应选择哪条 EXTI 线？(A)

A、 EXTI0　B、 EXTI1　C、 EXTI2 D、 EXTI15

解析： GPIOx0 引脚对应 EXTI0，PA0→EXTI0，PB0→EXTI0（需通过 AFIO 复用）。

5、中断嵌套是指？(B)

A、多个中断同时发生　B、高优先级中断打断低优先级中断

C、中断服务函数调用其他函数　D、中断屏蔽

解析：高抢占优先级的中断可以打断正在执行的低优先级中断，形成嵌套。

6、在 EXTI 的 HAL 库中，HAL_GPIO_EXTI_Callback 函数的作用是（ C ）

A、 作为所有外部中断发生后的通用处理函数

B、 获取 EXTI 的句柄

C、 外部中断回调函数，用于处理具体的中断服务

D、 产生一个软件中断

解析：HAL_GPIO_EXTI_Callback 函数是外部中断回调函数，用于处理具体的中断服务；作为所有外部中断发生后的通用处理函数的是 HAL_GPIO_EXTI_IRQHandler；获取 EXTI 句柄的函数是 HAL_EXTI_GetHandle；产生软件中断的函数是 HAL_EXTI_GenerateSWI。

7、STM32 中，中断优先级的分组方式有几种？(B)

A、 3　B、 5　C、 7　D、 9

解析：优先级分组有 5 种（如 Group0~Group4），决定抢占优先级和子优先级的位数分配。

8、EXTI 的“双边沿触发”是指？(C)

A、仅高电平时触发　B、仅低电平时触发

C、上升沿和下降沿均触发　D、电平持续期间触发

解析：双边沿触发会在引脚电平从高到低或低到高变化时均触发中断

9、EXTI15_10_IRQn 表示？(D)

A、EXTI 线 15 的中断号　　B、EXTI 线 10 的中断号

C、外部中断的总线号　　D、EXTI 线 10~15 共享的中断号

解析：EXTI10 ~ EXTI15 共享同一个中断向量（IRQn），需在回调函数中通过引脚号区分具体中断源。

10、HAL 库中，清除 EXTI 中断标志位的函数是？(B)

A、HAL_GPIO_WritePin　　B、 __HAL_GPIO_EXTI_CLEAR_FLAG

C、HAL_NVIC_ClearPendingIRQ　　D、HAL_EXTI_Reset

解析：*__HAL_GPIO_EXTI_CLEAR_FLAG(GPIO_PIN)* 用于清除特定 EXTI 线的中断标志。

11、以下哪种场景适合使用外部中断？(B)

A、周期性读取温度传感器　　B、检测按键按下事件

C、循环点亮 LED　　D、串口数据发送

解析：外部中断适合处理异步事件（如按键按下），可实时响应，避免轮询消耗 CPU 资源。

12、STM32 中，中断优先级数值越小表示？(B)

A、优先级越低　B、优先级越高　C、无影响　D、随机分配

解析：优先级数值越小，优先级越高（如优先级 0 比 1 更高）。

13、当多个中断源同时提出中断请求时，STM32 首先判断（ A ）。

A、 抢占优先级大小　　B、 中断向量表顺序

C、 子优先级大小　　D、 中断号大小

解析：当多个中断源同时提出中断请求时，STM32 先判断抢占优先级的大小，如果抢占优先级相同，则比较响应优先级（子优先级）的大小，若抢占优先级和响应优先级均相同，则根据中断向量表中的顺序来决定。

14、STM32 中断优先级分组中，NVIC_PriorityGroup_2 的抢占优先级取值范围是（ B ）

A、 0 - 1　　B、 0 - 3　　C、 0 - 7　　D、 0 - 15

解析：NVIC_PriorityGroup_2 的抢占优先级取值范围是 0 - 3，子优先级取值范围是 0 - 3。

15、在 EXTI 的 HAL 库应用实例中，按键中断触发后，首先会调用（ C ）。

A、HAL_GPIO_EXTI_Callback 函数　　B、用户自定义中断函数

C、HAL_GPIO_EXTI_IRQHandler 函数　　D、EXTI4_IRQHandler 函数

解析：外部的按键中断触发后，首先会调用 GPIO 相应引脚的外部中断处理函数 HAL_GPIO_EXTI_IRQHandler。

16、在 EXTI 的 HAL 库中，用于设置外部中断 EXTI 的外部中断线路的函数是（ A ）

A、HAL_EXTI_SetConfigLine　　B、HAL_EXTI_GetConfigLine

C、HAL_EXTI_ClearConfigLine　　D、HAL_EXTI_GetHandle

解析：HAL_EXTI_SetConfigLine 函数用于设置外部中断 EXTI 的外部中断线路；HAL_EXTI_GetConfigLine 用于获取已配置好的外部中断 EXTI 的外部中断线路；HAL_EXTI_ClearConfigLine 用于将已设置好的外部中断线路清除；HAL_EXTI_GetHandle 用于获取 EXTI 的句柄。

二、简答题

1、为什么需要清除 EXTI 中断标志位？

参考答案：防止中断重复触发。标志位未清除时，硬件会持续认为有中断请求。

2、STM32 的中断优先级有抢占式优先级和响应优式先级两种。若系统中存在多个中断，一般遵循什么原则判断中断优先级？

参考答案：（1）高抢先优先级的中断可以打断低抢先优先级的中断服务，构成中断嵌套；

（2）中断优先级的数值越小，优先级级别越高；

（3）抢占优先级的优先级总是高于响应优先级；

（4）中断优先级判断：先判断抢占优先级的大小，如果抢占优先级相同，则比较响应优先级的大小，若抢占优先级和响应优先级均相同，则根据中断向量表中的顺序来决定；

（5）Reset NMI Hard Fault 的优先级为负，且不可修改，高于普通的中断优先级。

3、中断服务函数中为何要避免耗时操作？

参考答案：长时间占用 CPU 会导致其他中断无法及时响应，降低系统实时性。

4、简述 GPIO 与 EXTI 外部中断的连接方式及特点。

答：GPIO 端口的 PA0~PG0 连接到 EXTI0，PA1~PG1 连接到 EXTI1，依此类推，直至 PA15~PG15 连接到 EXTI15。即 PAx~PGx 端口的中断事件都连接了 EXTIx，同一时刻 EXTIx 只能响应一个端口的事件触发，不能够同一时间响应所有 GPIO 的事件，但可以分时复用。

5、在使用 EXTI 的 HAL 库函数配置外部中断时，HAL_GPIO_EXTI_IRQHandler 和 HAL_GPIO_EXTI_Callback 函数分别起什么作用，它们是如何协作的？

参考答案：HAL_GPIO_EXTI_IRQHandler 函数是外部中断的通用处理函数，用于处理中断标志位清除等基础操作，并根据中断引脚调用 HAL_GPIO_EXTI_Callback 函数。HAL_GPIO_EXTI_Callback 函数是用户自定义的中断处理函数，用于编写具体的中断服务代码，实现特定功能。当中断发生时，先执行 HAL_GPIO_EXTI_IRQHandler 函数，它识别中断引脚后调用 HAL_GPIO_EXTI_Callback 函数，用户在该函数中编写的代码得以执行。

二、其它

1、设计一个基于 STM32 的智能安防系统，利用 EXTI 中断实现人体红外检测报警功能。要求：描述硬件连接、软件配置步骤，并编写关键代码片段。

已知硬件连接：人体红外传感器的输出引脚连接到 STM32 的 PA1 引脚，蜂鸣器连接到 PB5 引脚，用于报警。

参考答案：

（1）软件配置步骤：使用 STM32CubeMX 配置 PA1 为外部中断模式，选择合适的触发方式（如上升沿触发）；配置 PB5 为通用输出模式；在 NVIC 中使能 PA1 对应的 EXTI 中断，并设置合适的优先级；生成代码后，在 HAL_GPIO_EXTI_Callback 函数中编写蜂鸣器报警代码。

（2）关键代码片段：

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{     if(GPIO_Pin == GPIO_PIN_1)
      {          HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_SET);
                     // 蜂鸣器响
                 HAL_Delay(2000); // 持续 2 秒
                HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_RESET);
                // 蜂鸣器停止     }
}
```

2、学习板上 LED 灯和按键的硬件连接如图 1 和图 2 所示，要求通过编程来控制 LED 灯的亮灭。请完成以下问题。

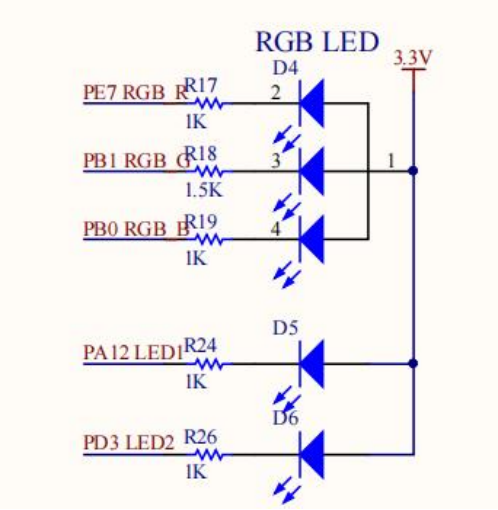

图 1

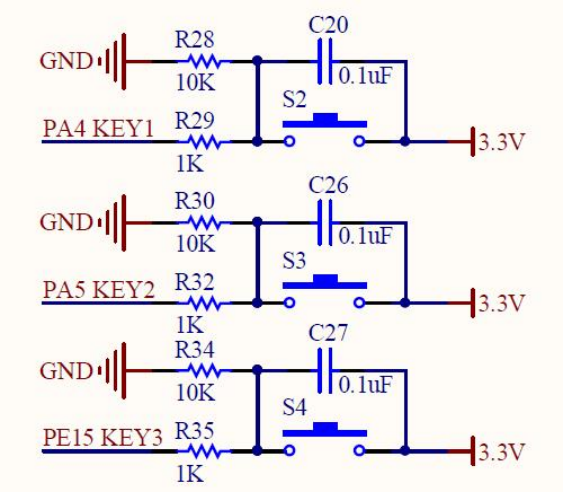

图 2

（1）如图 2 所示，LED2 灯的控制引脚是连接在 GPIO 的哪个引脚上，当引脚输出高电平时，LED2 灯是亮还是灭？（2 分）

答：连接在 PD3 引脚上，LED2 灯灭。

（2）如图 3 所示，按键 KEY1 的控制引脚是连接在 GPIO 的哪个引脚上？对按键引脚进行 GPIO 初始化时，应把按键设置成什么输入模式？（2 分）

答：连接在 PA4 引脚上，配置成下拉或者浮空输入模式。

（3）以下①~⑥是基于 HAL 库的函数声明，请写出调用对应函数时可以实现的功能。（12 分）

①void HAL_GPIO_Init(GPIO_TypeDef *GPIOx, GPIO_InitTypeDef *GPIO_Init);

功能描述：GPIO 初始化函数

②void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState);

功能描述：设置选定的端口引脚输出高电平或低电平

③GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);

功能描述：读取选定的端口引脚的电平状态

④void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);

功能描述：设置选定端口引脚的电平状态翻转

⑤HAL_StatusTypeDef HAL_TIM_Base_Start_IT (TIM_HandleTypeDef *htim);

功能描述：　　在中断方式下启动定时器

⑥void HAL_TIM_PeriodElapsedCallback (TIM_HandleTypeDef *htim);

功能描述：　定时器定时/更新中断回调函数，用户在该函数内编写实际的中断处理程序

（4）要求通过编程实现：当按键 KEY1 被按下时，LED2 灯亮；当按键松开时，LED2 灯灭。在使用 STM32CubeMX 创建了 KEIL5 工程的前提下，请选用①~⑥函数编写用户程序，在⑦~⑧处将代码补充完整，实现该功能。（4 分）

```
while（1）
{       if ( ⑦ HAL_GPIO_ReadPin(GPIOA,GPIO_PIN_4 ) ==
GPIO_PIN_RESET)
          {
              HAL_Delay(3);
              HAL_GPIO_WritePin(GPIOD,GPIO_PIN_3,GPIO_PIN_RESET);
           }
else     /* 反之 LED2 灯熄灭 */
     {
        ⑧ HAL_GPIO_WritePin(GPIOD,GPIO_PIN_3,GPIO_PIN_SET);
     }
}
```

（5）下面这段代码是外部中断的通用处理函数，请在⑨~⑩处对代码进行注释。（4 分）

```
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
     if (__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != 0x00u)
 // ⑨ 判断某个线上的中断是否发生
        {
              __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
 // ⑩   清除中断标志
```

```
            HAL_GPIO_EXTI_Callback(GPIO_Pin);
            }
    }
```

单片机--第 6 章 定时器练习题

主要考查的知识点：定时器的种类、计数模式、定时周期（即溢出时间）的计算、TIM 的常用 HAL 库接口函数有哪些？对应实现什么功能。

一、单选题

1、定时器本质上属于（ B ）。

A、 振荡器　　B、 计数器　　C、 寄存器　　D、 存储器

解析：定时器和计数器本质上都是计数器，定时器是对周期信号计数的特殊计数器，用于实现定时功能 。

2、STM32 中通用定时器 TIM3 的挂接总线是（ A ）。

A、 APB1　　B、 APB2　　C、 AHB　　D、 以上都不对

解析：STM32 的通用定时器 TIM2 - TIM7 挂接在 APB1 总线上，TIM1 和 TIM8 挂接在 APB2 总线上。

3、以下哪种不是 STM32 通用定时器的计数模式（ C ）。

A、向上计数　B、向下计数　　C、向左计数　　D、中心对齐计数

4、在向上计数模式下，STM32 通用定时器计数器溢出时，计数值会（ C ）。

A、 保持不变　　B、 变为最大值

C、 重新从 0 开始　　D、 变为自动重载值

解析：向上计数模式下，计数器从 0 计数到自动加载值（TIMx_ARR）后溢出，然后重新从 0 开始计数并产生溢出事件。

5、若 STM32 定时器时钟 CK_PSC 为 72MHz，预分频系数 PSC 为 71，自动重载值 ARR 为 999，其定时时间 T 为（ C ）。

A、 1s　　B、 100ms　　C、 1ms　　D、 0.1ms

6、定时器 HAL 库接口函数的声明在（ B ）文件中。

A、 stm32f1xx_hal_tim.c　　B、 stm32f1xx_hal_tim.h

C、 main.c　　D、 system_stm32f1xx.c

解析：定时器 HAL 库接口函数的源码在 stm32f1xx_hal_tim.c 中，其对应的头文件 stm32f1xx_hal_tim.h 声明了 TIM 所有的库函数。

7、用于在轮询方式下启动定时器的 HAL 库函数是（ A ）。

A、 HAL_TIM_Base_Start（）　　B、 HAL_TIM_Base_Init（）

C、 HAL_TIM_Base_Start_IT（）　　D、 HAL_TIM_Base_Stop（）

解析：HAL_TIM_Base_Start 函数用于在轮询方式下启动定时器；HAL_TIM_Base_Init 用于初始化时基单元；HAL_TIM_Base_Start_IT 用于在中断方式下启动定时器；HAL_TIM_Base_Stop 用于停止定时器。

8、清除定时器中断标志的宏是（ C ）。

A、 __HAL_TIM_GET_COUNTER　　B、 AL_TIM_PeriodElapsedCallback

C、 __HAL_TIM_CLEAR_IT　　D、 HAL_TIM_IRQHandler

解析：__HAL_TIM_CLEAR_IT 用于清除定时器的各类中断标志；
__HAL_TIM_GET_COUNTER 用于获得定时器的计数值；
HAL_TIM_PeriodElapsedCallback 是定时器更新中断回调函数；
HAL_TIM_IRQHandler 是定时器中断发生后的通用处理函数。

9、在 STM32CubeMX 中配置定时器 TIM3 产生 1s 定时中断，若预分频系数设置为 36000 - 1，自动重载值应设置为（ B ）。

A、 1000 - 1　　B、 2000 - 1　　C、 3000 - 1　　D、 4000 - 1

10、STM32CubeMX 中配置定时器时钟源应选择？（ B ）

A、 External Clock　　B、 Internal Clock

C、 ETR Clock　　D、 ITR Clock

解析：默认使用内部时钟（Internal Clock）。

11、通用定时器 TIM3 的捕获 / 比较通道数量是（ C ）。

A、 2 个　　B、 3 个　　C、 4 个　　D、 5 个

12、在定时器 HAL 库中，用于处理定时器中断具体任务的回调函数是（ B ）。

A、 HAL_TIM_IRQHandler（）　　B、HAL_TIM_PeriodElapsedCallback（）

C、 HAL_TIM_Base_Start_IT（）　　D、 HAL_TIM_Base_Init（）

解析：HAL_TIM_PeriodElapsedCallback 是回调函数，用来处理具体的中断任务，由定时器中断通用处理函数 HAL_TIM_IRQHandler 调用 。

13、若要读取定时器 TIM3 的当前计数值，应使用的宏是（ C ）。

A、 __HAL_TIM_CLEAR_IT (&htim3, TIM_IT_UPDATE)

B、 HAL_TIM_PeriodElapsedCallback (&htim3)

C、 __HAL_TIM_GET_COUNTER (&htim3)

D、 HAL_TIM_IRQHandler (&htim3)

解析：__HAL_TIM_GET_COUNTER 宏用于获得定时器的计数值，传入定时器句柄即可获取对应定时器的当前计数值。

14、STM32 中基本定时器 TIM6 的计数方式是（ A ）。

A、 向上计数　　B、 向下计数

C、 向上 / 向下计数　　D、 中心对齐计数

解析：STM32 的基本定时器 TIM6 和 TIM7 只能向上计数。

15、假如要设置定时时间为 500ms，并配置中断，CK_INT 按照系统默认初始化来（即 72MHz），PSC 取 7199，则 ARR 取值应为多少？（ B ）

A、 499　　B、 4999　　C、 500　　D、 5000

16、在定时器中断处理中，HAL_TIM_IRQHandler 函数由（ B ）自动生成。

A、 用户手动编写　　B、 STM32CubeMX 软件

C、 Keil 软件　　D、 IAR 软件

解析：HAL_TIM_IRQHandler 函数是所有定时器中断发生后的通用处理函数，由 STM32CubeMX 软件自动生成。

17、定时器溢出中断的标志位在哪个寄存器中？（ B ）

A、 TIMx_CR　　B、 TIMx_SR　　C、 TIMx_DIER　　D、 TIMx_ARR

解析：状态寄存器 TIMx_SR 记录中断标志位。

18、自动重载寄存器 TIMx_ARR 的作用是？（ B ）

A、 设置预分频系数　　B、 设置计数终值

C、 清除中断标志　　D、 选择时钟源

解析：TIMx_ARR 用于设置计数器的终值或初值。

19、预分频寄存器 TIMx_PSC 的取值范围是？(A)

A、 0~65535　　B、 0~255　　C、 1~1024　　D、 固定为 16

解析：16 位定时器的 PSC 范围为 0 ~ 65535。

20、SM32F103VE 芯片的定时器，属于高级定时器的是（ C ）

A、TIM1、TIM2　　B、TIM3、TIM4

C、TIM1、TIM8　　D、TIM6、TIM7

二、判断题

1、定时器只能用于产生固定时长的定时，不能用于其他功能。（ x ）

解析：定时器除了定时功能，还可用于测量脉冲宽度、生成 PWM 波等多种功能。

2、STM32 的高级定时器和通用定时器都可以工作在中心对齐计数模式。（√ ）

解析：STM32 的高级定时器 TIM1、TIM8 和通用定时器 TIM2 - TIM5 都支持向上计数、向下计数和中心对齐计数模式。

3、定时器的预分频系数越大，计数时钟频率越高。（ x ）

解析：预分频系数 PSC 越大，计数时钟频率越低。

4、HAL_TIM_Base_Start_IT 函数可以在定时器未初始化时直接调用。（ x ）

解析：HAL_TIM_Base_Start_IT 函数需在定时器初始化完成之后由用户调用，且中断启动前需清除在时基单元初始化过程中产生的中断标志。

5、基本定时器 TIM7 没有捕获 / 比较通道，不能用于产生 PWM 波。（√ ）

解析：基本定时器 TIM6 和 TIM7 都没有捕获 / 比较通道，无法直接用于产生 PWM 波 。

6、__HAL_TIM_GET_COUNTER 宏不仅可以获取定时器当前计数值，还能修改计数值。（ x ）

解析：__HAL_TIM_GET_COUNTER 宏只能获取定时器的计数值，不能修改计数值。

7、STM32 通用定时器的计数寄存器和自动重载寄存器都是 16 位的。（√ ）

解析：STM32 通用定时器如 TIM2 - TIM5 的计数寄存器和自动重载寄存器都是 16 位的，取值范围为 0 - 65535。

8、定时器的定时时间与系统时钟频率无关，只取决于预分频系数和自动重载值。（ x ）

9、在 STM32CubeMX 中配置 GPIO 引脚为定时器功能时，不需要配置引脚的复用功能。（ x ）

解析：当 GPIO 引脚用于定时器功能时，需要配置为复用功能，使能定时器与该引脚的连接。

10、通用定时器 TIM3 和 TIM5 的所有功能和特性完全相同。（ x ）

解析：通用定时器 TIM3 和 TIM5 基本功能相似，但在具体应用场景中，如引脚分配、中断优先级等方面可能存在差异。

11、在定时器中断处理中，清除中断标志的操作可以防止中断的重复触发。
（ √ ）解析：清除中断标志后，定时器不再满足中断触发条件，可防止中断的重复触发，确保中断处理的准确性。

12、在 STM32CubeMX 中生成工程代码后，不能再修改定时器的配置参数。
（ x ）解析：在 STM32CubeMX 中生成工程代码后，可重新打开工程修改定时器配置参数，重新生成代码后即可应用新的配置。

13、向下计数模式下，计数器从 0 开始计数。（ x ）

解析： 向下计数从 ARR 值开始，减到 0。

14、自动重载寄存器 TIMx_ARR 的值必须大于 PSC。（ x ）

解析：ARR 和 PSC 无直接大小关系，ARR 决定计数终值。

15、STM32CubeMX 生成的代码可直接使用，无需修改。（ x ）

解析：需用户添加中断回调函数等逻辑。

16、HAL_Delay()函数基于 SysTick 定时器实现。（ √ ）

解析：HAL 库的延时函数依赖 SysTick。

17、定时器中断优先级越高，响应速度越快。（ √ ）

解析：高优先级中断会抢占低优先级。

18、定时器中断中可调用耗时长的函数。（ x ）

解析：中断服务函数应尽量简短，避免影响系统实时性。

19、定时器溢出频率公式为 f = CK_PSC / (ARR×PSC)。（ x ）

解析：正确公式为 f = CK_PSC / [(ARR+1)(PSC+1)]。

20、预分频器的作用是扩大定时器的定时范围，并获取精确的时钟。（ √ ）

三、简答题

1、简述 HAL 库中配置定时器中断的步骤。

参考答案：

（1）调用 HAL_TIM_Base_Init()初始化定时器。
（2）使用 HAL_TIM_Base_Start_IT()启动中断。
（3）实现 HAL_TIM_PeriodElapsedCallback()回调函数。

2、定时器中断与轮询方式有何区别？

中断方式： 异步响应，不占用 CPU 时间，适合实时任务。
轮询方式： 需主动查询标志位，占用 CPU 资源，适合简单应用。

3、为什么定时器中断服务函数中需清除中断标志？

参考答案： 清除标志位是为了避免重复进入中断，标志位由硬件置 1，需软件清零。

4、简述 STM32 通用定时器的定时原理，并说明定时时间的计算方法。

参考答案：（1）STM32 通用定时器的定时原理是通过对时钟信号进行计数来实现定时。定时器内部有预分频模块、计数模块和自动重载模块。预分频模块对输入的时钟信号进行分频，得到计数时钟 CK_CNT。计数模块中的核心计数器对 CK_CNT 进行计数，每来一个计数时钟脉冲，计数值加 1 或减 1（根据计数模式而定）。当计数值达到自动重载寄存器（TIMx_ARR）设定的值时，产生溢出事件，可触发中断或其他操作，从而实现定时功能。

（2）定时时间的计算公式为 T = (ARR+1)(PSC+1)/CK_PSC，其中 T 为定时时间，ARR 为自动重载值，PSC 为预分频系数，CK_PSC 为预分频时钟频率，一般为 72MHZ。

第 6 章--PWM 和输入捕获

主要考查的知识点：PWM 的基本原理，PWM 的周期计算公式、占空比的影响因素、输入捕获的工作过程、PWM 的常用 HAL 库接口函数有哪些？对应实现什么功能。

一、单选题

1、PWM 的中文全称是？ （ B ）

A、 脉冲频率调制　　B、 脉冲宽度调制

C、 脉冲幅度调制　　D、 脉冲相位调制

2、PWM 占空比的计算公式是？ （ A ）

A、高电平时间/周期 × 100%　　B、 低电平时间 / 周期 × 100%

C、高电平时间/低电平时间 × 100%　　D、周期/高电平时间 × 100%

解析：占空比定义为高电平时间占周期的百分比，即占空比 = Tp/T * 100% 。

3、STM32 定时器中，决定 PWM 周期的参数是？（ D ）

A、 CCRx　　B、 ARR　　C、 PSC　　D、 B 和 C 共同决定

解析：PWM 周期公式为 T = (ARR+1)(PSC+1)/TIMxCLK ，由自动重装载值（ARR）和预分频系数（PSC）共同决定。

4、HAL_TIM_PWM_Start( ) 函数的作用是？（ B ）

A、初始化 PWM 硬件资源　　B、启动 PWM 信号输出（轮询方式）

C、设置捕获 / 比较寄存器值　　D、停止 PWM 输出

解析：该函数用于在轮询方式下启动 PWM 波形输出，用户需在初始化后调用。

5、STM32 定时器输入捕获功能可以捕获什么信号？（ C ）

A、 仅上升沿　　B、 仅下降沿

C、 上升沿、下降沿或双边沿　　D、 固定电平信号

解析：输入捕获支持设置捕获边沿为上升沿、下降沿或双边沿（通过 `__HAL_TIM_SET_CAPTUREPOLARITY` 函数）。

6、以下哪个函数用于设置 PWM 的占空比？（ C ）

A、 HAL_TIM_PWM_Init（ ）　　B、 HAL_TIM_PWM_Start（ ）

C、 __HAL_TIM_SET_COMPARE（ ）　　D、 HAL_TIM_IC_Init（ ）

解析：`__HAL_TIM_SET_COMPARE` 直接设置捕获 / 比较寄存器（CCR）的值，从而控制占空比。

7、STM32 定时器输入捕获的步骤顺序是？（ A ）

A、 滤波→边沿检测→分频→捕获　　B、 边沿检测→滤波→分频→捕获

C、 分频→滤波→边沿检测→捕获　　D、 滤波→分频→边沿检测→捕获

解析：输入捕获过程为：检测边沿→滤波→判断边沿类型→分频处理→触发中断。

8、HAL_TIM_IC_Start_IT( ) 函数的作用是？（ B ）

A、 初始化输入捕获硬件　　B、 启动中断方式的输入捕获

C、 停止中断方式的输入捕获　　D、 读取捕获值

解析：该函数用于在中断模式下启动输入捕获，使能捕获中断并启动定时器。

9、输入捕获实验中，捕获值存储在哪个寄存器？（ C ）

A、 TIMx_ARR　　B、 TIMx_PSC

C、 TIMx_CCRx　　D、 TIMx_CR1

解析：捕获值存储在捕获 / 比较寄存器（TIMx_CCRx），通过 `HAL_TIM_ReadCapturedValue` 读取。

10、HAL_TIM_IC_CaptureCallback( )函数的作用是？（ B ）

A、 初始化输入捕获　　B、 处理捕获中断后的任务

C、 启动输入捕获　　D、 设置捕获边沿

解析：该函数是输入捕获中断的回调函数，用户需在此编写中断处理逻辑。

11、输入捕获实验中，如何实现边沿触发极性的切换？（ B ）

A、 调用 HAL_TIM_SetCapturePolarity（）函数

B、 调用__HAL_TIM_SET_CAPTUREPOLARITY （）函数

C、 重新初始化定时器

D、 修改 PSC 和 ARR 参数

解析：通过宏 `__HAL_TIM_SET_CAPTUREPOLARITY()` 设置捕获边沿（上升沿 / 下降沿）。

12、以下哪个参数影响 PWM 的占空比？（ C ）

A、 ARR　　B、 PSC　　C、 CCRx　　D、 以上都是

解析：占空比公式为 `CCRx/(ARR+1)×100%`，直接由 CCRx 决定，ARR 影响周期。

13、每个定时器最多有多少个独立 PWM 通道？（ D ）

A、 1　　B、 2　　C、 3　　D、 4

二、判断题

1、PWM 可以通过数字输出来控制模拟电路。（ √ ）

解析：PWM 是利用微处理器数字输出来控制模拟电路的技术（如电机调速、LED 亮度）。

2、占空比为 50% 的 PWM 波形，高电平和低电平时间相等。（ √ ）

解析：占空比定义为高电平时间占周期的比例，50% 即高低电平时间各占一半。

3、STM32 定时器的每个通道可以独立配置为 PWM 输出或输入捕获。（ √ ）

解析：每个定时器的 1~4 个通道可独立选择功能（输出比较 / PWM 或输入捕获）。

4、HAL_TIM_PWM_Init 函数由用户手动编写，不能通过 CubeMX 生成。（ x ）

解析：该函数由 CubeMX 自动生成，用户无需手动编写。

5、PWM 的周期越大，频率越高。（ x ）

解析：频率是周期的倒数，周期越大，频率越低。

6、输入捕获实验中，需要将 GPIO 引脚配置为输出模式。（ x ）

解析：输入捕获引脚需配置为输入模式（如 PA7 配置为 TIM3_CH2 输入捕获）。

7、HAL_TIM_IC_Stop_IT( ) 函数用于停止输入捕获，并禁止中断。（ √ ）

解析：该函数停止中断方式的输入捕获，关闭通道并禁止中断。

8、输入捕获的滤波功能可以消除高频噪声，提高信号检测的准确性。（ √ ）

解析：输入捕获前需进行滤波，防止噪声触发错误的边沿检测。

9、在 PWM 输出实验中，定时器的计数模式只能是向上计数。（ √ ）

解析：计数模式可配置为向上、向下或中央对齐

10、输入捕获可以测量脉冲的宽度。（ √ ）

三、简答题

1、关于 STM32 的定时器 PWM 输出，请回答以下问题。

（1）简述 PWM 输出工作原理；

（2）PWM 的周期 T 计算公式和各参数的含义；

（3）PWM 的占空比受哪些因素影响。

答：（1）每个定时器有四个通道，每个通道都有一个 CCRx 捕获比较器(x 为通道号，比如通道 1 的比较寄存器为 CCR1)，将比较寄存器(TIMx_CCRx)和计数器(TIMx_CNT)的值比较，通过比较结果输出高低电平，实现 PWM 信号输出。

（2）PWM 的周期 T = (ARR+1)(PSC+1)/CK_PSC，ARR：自动重装载值；

PSC：预分频系数；CK_PSC：定时器时钟频率。

（3）PWM 的占空比= CCRx/(ARR+1)*100%，受 CCRx 和 ARR 影响。

四、其它

1、编写代码：使用 TIM3_CH1 输出占空比为 30% 的 PWM 波，已知 ARR=499，PSC=71（系统时钟 72MHz）。

参考答案：

```
// 计算 CCR 值：占空比 30% = CCR/(499+1) → CCR=150
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 150);
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1); // 启动 PWM 输出
```

解析：占空比公式为 CCR/(ARR+1) ，ARR=499 时周期为 500，30% 占空比对应 CCR=150。

2、如下图 3 所示，LED 灯连接在 PE5 引脚上，请完成以下问题。

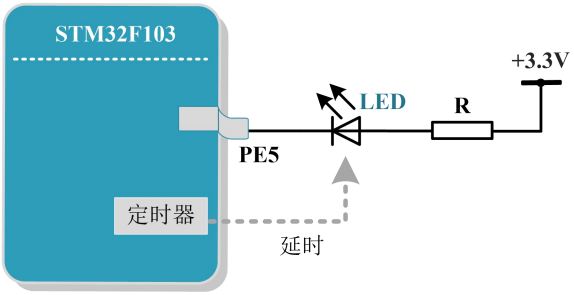

图 3

（1）试简述 LED 灯亮灭的工作原理。

答：因为 LED 灯的控制引脚连接在 PE5 上，另一端通过电阻 R 连接在 3.3V 的电源端，所以当 PE5 引脚输出高电平时灯灭，PE5 引脚输出低电平时灯亮。同时可以通过定时器产生 PWM 脉冲波形，输出占空比可变的 PWM 波驱动 LED 灯。

（2）以下①~③是基于 HAL 库的函数声明，请写出调用对应函数时可以实现的功能。

①HAL_StatusTypeDef HAL_TIM_Base_Init(TIM_HandleTypeDef *htim);

功能描述：定时器初始化函数，根据定时器句柄中设定的参数初始化时基单元

②HAL_StatusTypeDef HAL_TIM_Base_Start _IT(TIM_HandleTypeDef *htim);

功能描述：在中断方式下启动定时器

③void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);

功能描述：设置选定端口引脚的电平状态翻转

（3）利用定时器 TIM4 产生 1s 的定时中断，然后在中断回调函数中控制 LED 灯，实现 LED 的状态取反（闪烁）。在使用 STM32CubeMX 创建了 KEIL5 工程的前提下，请选用①~③函数编写用户程序，在④-⑤处将代码补充完整，实现该功能。

在 main.c 文件中的/*USER CODE BEGIN2 */和/*USER CODE END2 */之间添加开启定时器 TIM4 中断的程序

```
MX_TIM4_Init();
/* USER CODE BEGIN 2 */
④ HAL_TIM_Base_Start_IT(&htim3);    //中断方式开启定时器 3
/* USER CODE END 2 */
```

在 mian.c 文件的/*USER CODE BEGIN4 */和/*USER CODE END4 */之间添加 TIM4 的中断回调函数，实现 LED 灯的状态取反（闪烁）。

```
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Instance == htim4.Instance)
    {
        ⑤HAL_GPIO_TogglePin ( GPIOE , GPIO_PIN_5 );
            //LED 灯的状态取反
    }
}
/* USER CODE END 4 */
```